

HealthClaimHardened

Z7 implementation and validation report | 4 October 2026

RESULT: PASS - isolated hardened circuit, test suite, reproducible scripts, and benchmarks completed.

Executive summary

Created a separate HealthClaimHardened v1 Go module at `zkp/healthclaimhardened`. It composes the frozen Core relation with a canonical context commitment binding the exact request, deployment, policy version, registry-root version, authority key, nullifier, and claim amount. HealthClaimCore-v1 remains unchanged. No Gateway, Fabric, IPFS, Docker, networking, or federated-learning work was performed.

Validation snapshot

Measure	Result
Toolchain / cryptography	Go 1.25.8; gnark v0.16.3; gnark-crypto v0.21.0; Groth16 on BN254
Constraint system	22,021 R1CS constraints; 13 public variables; 31 private variables
Security tests	All passed: six top-level test functions and 37 nested cases
Benchmarks, 10 iterations	Proof generation 69.72 ms/op; verification 0.638 ms/op
Serialized Groth16 proof	164 bytes (measured with <code>gnark Proof.WriteTo</code>)
Environment	Linux amd64; AMD Ryzen 7 PRO 8840HS; same host/method as recorded Z6 run

Circuit statement

The circuit proves an authority-signed record is in the supplied Poseidon2 Merkle tree and satisfies the policy predicates: diagnosis membership in a fixed four-entry set, a 64-bit lab value within bounds, a coverage ceiling no greater than the policy maximum, and a public claim amount no greater than that ceiling. Merkle selectors are Boolean and the record index is derived from the path directions.

Context commitment

ContextCommitment = Poseidon2(tag=106, ProtocolDomain, Challenge, RecipientID, DeploymentID, PolicyVersion, RootVersion, PolicyCommitment, RegistryRoot, AuthorityKeyX, AuthorityKeyY, Nullifier, ClaimAmount). The tuple order is fixed and each input is a separate canonical BN254 scalar field element; strings are not concatenated. ProtocolDomain is constrained to the fixed protocol ID 202610041. Existing Core domain tags 101-105 remain dedicated to secret, record, policy, Merkle, and nullifier hashes.

Inputs and trust boundary

Public inputs (13)

RegistryRoot, RootVersion, PolicyCommitment, PolicyVersion, ClaimAmount, AuthorityKeyX, AuthorityKeyY, Nullifier, ProtocolDomain, Challenge, RecipientID, DeploymentID, ContextCommitment.

Private witness (31)

Diagnosis, LabValue, PolicyID, CoverageCeiling, PatientSecret, LabMin, LabMax, PolicyMaxClaim, four covered diagnoses, EdDSA signature (R,S), eight Merkle siblings, and eight Merkle direction bits.

The policy commitment continues to cover PolicyID, lab minimum and maximum, policy maximum claim, and the ordered four-diagnosis set. PolicyVersion is separately included in the context commitment. Clinical diagnosis, lab values, patient secret, signature, and Merkle path remain private.

Verifier responsibilities

The proof does not make public inputs trusted. A verifier must authenticate the issuer key and status; confirm the root/version and policy commitment/version against authoritative registry state; check recipient and deployment/channel; and compare the challenge with one it issued. Issuer-key registration must reject identity and enforce the intended prime-order subgroup.

Freshness and replay

The circuit proves binding to the exact challenge, but it cannot determine whether the challenge is fresh, unexpired, or unused. The verifier must generate an unpredictable challenge and enforce its acceptance window and one-time use. Root/policy validity and expiry are external checks; no wall-clock timestamp is constrained in-circuit.

Nullifier = Poseidon2(tag=105, PatientSecret, RecordIndex), with RecordIndex derived from Merkle direction bits. It is stable for that record across requests and is included in the context commitment. External UsedNullifiers state is still required to reject repeat claims.

Adversarial circuit tests

Each listed witness mutation was expected to make constraint solving fail; each test observed an unsatisfied witness. Full case names and properties are also recorded in SECURITY-TEST-MATRIX.md.

Mutation	Expected / observed	Property demonstrated
Challenge	Fail / passed	Exact challenge binding
Recipient	Fail / passed	Recipient binding
Protocol domain	Fail / passed	Fixed domain separation
Deployment context	Fail / passed	Deployment binding
Root version	Fail / passed	Versioned registry context
Policy version	Fail / passed	Versioned policy context
Registry root	Fail / passed	Root and path binding
Merkle sibling	Fail / passed	Authenticated membership
Policy commitment	Fail / passed	Recomputed policy
Nullifier	Fail / passed	Record nullifier binding
Authority key	Fail / passed	Signature-key binding

Mutation	Expected / observed	Property demonstrated
Authority signature	Fail / passed	Issuer authentication relation
Merkle direction = 2	Fail / passed	Boolean selector
Diagnosis not covered	Fail / passed	Policy membership
Lab below / above bounds	Fail / passed	Lab range checks
Claim above ceiling	Fail / passed	Claim ceiling
Coverage above policy maximum	Fail / passed	Policy maximum
Lab / claim / ceiling >64 bits	Fail / passed	Explicit range discipline
Root / policy version >64 bits	Fail / passed	Version width
Context commitment	Fail / passed	Canonical context recomputation

Proof-input mutation tests

For each case, the original valid proof was checked with a public witness containing one changed value. Verification was expected to reject and did reject all 13 mutations.

Changed public input	Property demonstrated
RegistryRoot	Proof cannot be reused against a different root.
RootVersion	Proof cannot be reused against another root version.
PolicyCommitment	Policy statement remains proof-bound.
PolicyVersion	Proof cannot be reused against another policy version.
ClaimAmount	Claim amount remains proof-bound.
AuthorityKeyX / AuthorityKeyY	Issuer key remains proof-bound.
Nullifier	Record nullifier remains proof-bound.
ProtocolDomain	Protocol domain is proof-bound and fixed in-circuit.
Challenge	Proof is bound to its exact challenge.
RecipientID	Proof is bound to the intended recipient.
DeploymentID	Proof is bound to deployment/channel context.
ContextCommitment	Committed context is proof-bound.

Additional validation

The deterministic Hardened-v1 fixture matched all public vectors. Context helper tests rejected negative/noncanonical field inputs and a policy version above 64 bits. A self-selected authority-key test intentionally remained satisfiable after signing and recomputing context; this confirms external key anchoring is required, rather than claiming the circuit supplies a trust anchor.

Z6 comparison

Same Linux amd64 host, CPU, Go and gnark versions, and 10-iteration benchmark methodology. Setup and witness preparation are outside timed sections. Z6 values are from the recorded baseline.

Metric	Core v1 (Z6)	Hardened v1 (Z7)	Delta
Constraints	19,657	22,021	+2,364 (+12.0%)
Public / private	6 / 31	13 / 31	+7 public; private unchanged
Proof generation	64.80 ms/op	69.72 ms/op	+4.92 ms (+7.6%)
Generation bytes/op	21,163,069	21,697,900	+534,831
Generation allocs/op	2,644	2,635	-9
Proof verification	0.635 ms/op	0.638 ms/op	+0.003 ms (+0.5%)
Verification bytes/op	30,384	33,130	+2,746
Verification allocs/op	165	183	+18
Proof serialization	Not measured	164 bytes	No direct Z6 comparison

Reproducibility and changed files

All three scripts completed successfully: scripts/setup.sh, scripts/test.sh, and scripts/benchmark.sh. ``go mod verify`` reported all modules verified. Commands require Go 1.25.8; this environment used `GOCACHE=/tmp/healthclaim-go-cache` because its default Go build cache is read-only.

New files are confined to `zkp/healthclaimhardened/`: `README.md`, `SECURITY-TEST-MATRIX.md`, `circuit.go`, `circuit_test.go`, `commitment.go`, `merkle.go`, `signature.go`, `go.mod`, `go.sum`, `scripts/setup.sh`, `scripts/test.sh`, `scripts/benchmark.sh`, and `testdata/hardened-v1-vectors.json`.

Remaining limitations and next step

No production key ceremony, trusted-registry lookup, challenge issuance/storage, expiry enforcement, used-nullifier storage, application mapping for recipient/deployment IDs, or Gateway/Fabric/IPFS integration exists. Merkle depth is eight and diagnosis set size is four. The next step is to design and review the verifier trust boundary before any system integration.

Z7 scope is complete. No Gateway, Fabric, IPFS, Docker, networking, or federated learning code was changed.